Uma Revisão Sistemática sobre o Gargalo de Von Neumann: Estratégias na Hierarquia de Memória Cache e Armazenamento Secundário

A Systematic Review on the Von Neumann Bottleneck: Strategies in Cache Memory Hierarchy and Secondary Storage

Anderson Vicente de Freitas[1], **André Hércules Gonçalves da Silva Pereira**[1], **Walter Felipe Dos Santos**[1]

[1]Colegiado De Ciencia Da Computação
Universidade Federal do Vale do São Francisco (UNIVASF) – Campus Salgueiro
Salgueiro – CEP: 56.000-000 – PE – Brasil

Abstract. *The von Neumann bottleneck is a limitation on the transfer rate between the processor and the main memory of a computer, in which the same transfer line is used for both instructions and data, a problem aggravated by the uneven evolution of the two components. A bibliographic review was therefore carried out using textbooks, teaching material and articles, selected and filtered according to strict criteria. The concepts found were organized into a taxonomy and represented in diagrams and tables, and the average memory access time model was described using hypothetical values. Multilevel caches, write policies, SSDs as HDD caches and virtual memory, with the risk of thrashing, are some of the strategies used to mitigate the von Neumann bottleneck. Future projections point to in-memory computing and hybrid systems to overcome the problem.*
Keywords *Von Neumann bottleneck, Memory hierarchy, Cache memory, Secondary storage, Computer architecture.*

Resumo. *O Gargalo de von Neumann é uma limitação na taxa de transferência entre o processador e a memória principal de um computador, no qual a mesma linha de transferência é utilizada tanto para instruções quanto para dados, problema agravado também pela evolução descompassada entre os dois componentes. Assim, foi realizada uma pesquisa bibliográfica através de livros-texto, material didático, artigos, selecionados e filtrados segundo critérios rígidos. Os conceitos encontrados foram organizados na forma de uma taxonomia, representados em diagramas e tabelas, e o modelo de tempo médio de acesso foi descrito usando valores hipotéticos. Caches multinível, políticas de escrita, SSD como cache para HDD e memória virtual com o risco de thrashing são algumas estratégias utilizadas para amenizar o Gargalo de von Neumann. Projeções futuras indicam computação em memória e sistemas híbridos para superar o problema.*
Palavras-Chave *Gargalo de von Neumann, Hierarquia de memória, Memória cache, Armazenamento secundário, Arquitetura de computadores.*

1. Introdução

A computação moderna tem origem em sua primeira geração na década de 1940, com a criação do ENIAC (Electronic Numerical Integrator and Computer), considerado o

primeiro computador digital eletrônico de propósito geral do mundo, que tinha como objetivo realizar cálculos complexos mais rápido que cálculos humanos com enfoque bélico na Segunda Guerra Mundial [Stallings 2010]. Sua programação, porém, era exaustiva por ser feita manualmente, por meio da reconexão de cabos e do ajuste de chaves [Miletto et al. 2014]. Esse processo exaustivo mais tarde seria otimizado com a criação do conceito de *Programa Armazenado*: no qual instruções e dados passaram a ficar na mesma memória, isso permitiu que as tarefas pudessem ser mudadas apenas rodando um software diferente [Stallings 2010]. Posteriormente, em 1945, John von Neumann iniciou o projeto do EDVAC (Electronic Discrete Variable Computer), essa ideia deu origem à Arquitetura de von Neumann: composta por memória principal, unidade lógica e aritmética (ULA), unidade de controle (UC), dispositivos de entrada e dispositivos de saída. Essa arquitetura é praticamente a base da construção da maioria dos computadores atuais [Tanenbaum e Austin 2013]. Esse modelo apresenta, contudo, uma limitação central, conhecida como o Gargalo de von Neumann: como instruções e dados utilizam o mesmo caminho de comunicação entre a CPU e a memória principal, faz com que a taxa de transferência entre esses dois componentes seja limitada, o que consequentemente pode afetar o desempenho geral da máquina. O problema se agrava à medida que os processadores evoluíram em ritmo muito superior em relação à memória ao longo dos anos [Couto 2017].

Este artigo apresenta uma revisão bibliográfica das estratégias da hierarquia de memória que atenuam o gargalo de von Neumann, da memória cache até o armazenamento secundário. Como contribuição, propõe tabelas comparativas e um diagrama da hierarquia, elaborados e adaptados pelos autores, além de discutir os *trade-offs* de latência, custo e capacidade. A Seção 2 descreve a Metodologia de Pesquisa; a Seção 3, a Taxonomia e a Sistematização Teórica; a Seção 4, a Análise Crítica e discussões sobre os Gargalos Arquiteturais; e a Seção 5 contém as Considerações Finais.

2. Metodologia

Este trabalho trata de uma revisão bibliográfica de natureza teórica e analítica referente à área de Arquitetura e Organização de Computadores. Por ter caráter estritamente bibliográfico, não há experimentação empírica no projeto: baseia-se exclusivamente na leitura, comparação e síntese da literatura, tendo como objetivo responder à seguinte questão norteadora: quais são as estratégias que minimizam o Gargalo de von Neumann e quais são os *trade-offs* envolvidos?

Os materiais de referência foram coletados e selecionados por meio do Google Acadêmico e da bibliografia da disciplina. Foram incluídos livros-texto [Stallings 2010, Tanenbaum e Austin 2013, Hennessy e Patterson 2017], materiais didáticos [Miletto et al. 2014, de Oliveira e Shin-Ting 2019, Couto 2017], além de artigos relacionados à hierarquia de memória, cache, armazenamento secundário, gerência de memória e sites sobre o contexto atual do gargalo de von Neumann [Maués e Nascimento 2025, Ferreira 2015, Maziero et al. 2015, de Souza e da Silva 2023, Hess 2025]. Foram incluídas as fontes que tratam explicitamente da arquitetura de von Neumann, da organização da memória, da cache ou do armazenamento secundário, enquanto foram excluídos os textos que mencionam a arquitetura de von Neumann sem entrar em detalhes sobre suas técnicas e

funcionamento.

Para a análise, foram seguidas as seguintes etapas: (i) seleção do material segundo os critérios mencionados; (ii) criação de uma taxonomia dos conceitos presentes nos materiais (Seção 3), desde a origem do gargalo até as estratégias de cache e de armazenamento secundário; (iii) demonstração do impacto do gargalo na latência usando o modelo do tempo médio de acesso (Seção 3.3), cujos valores apresentados são hipotéticos e ilustram ordens de grandeza, uma vez que não há medição empírica direta; e (iv) níveis da hierarquia de memória e comparação entre as estratégias e(Figura 2 e 3). Além disso, é feita uma discussão crítica sobre latência, custo, capacidade e os *trade-offs* envolvidos no processo de mitigação do Gargalo de von Neumann (Seção 4).

O texto foi redigido pelos autores. Ferramentas de inteligência artificial generativa foram utilizadas apenas para checagem gramatical do documento e para auxiliar no código LaTeX, conforme definido nas diretrizes da disciplina.

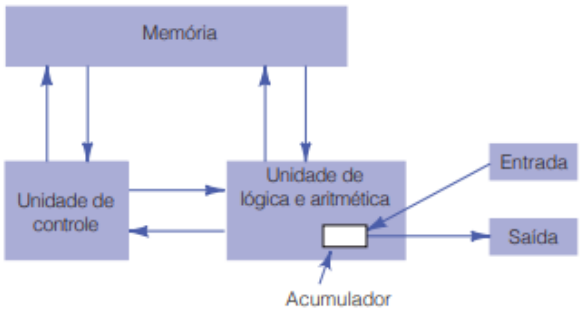
3. Taxonomia e Sistematização Teórica

3.1. A Arquitetura de Von Neumann e a Origem do Gargalo

A arquitetura de von Neumann baseia-se em três princípios: dados e instruções ficam em uma única memória de leitura e escrita; o conteúdo dessa memória é endereçável por posição, independentemente do tipo de dado; e a execução é sequencial, salvo desvio explícito [Stallings 2010]. Seus componentes são a memória principal, os dispositivos de entrada e saída e a CPU, formada pela ULA, pela unidade de controle e pelos registradores [Couto 2017, Tanenbaum e Austin 2013] (Figura 1). A unidade de controle busca as instruções uma a uma e coordena sua execução, no ciclo de busca, decodificação e execução [Couto 2017].

Como instruções e dados trafegam pelo mesmo canal entre a CPU e a memória, a taxa de transferência desse canal se limita a taxa com que o processador consegue operar, e a CPU precisa esperar pelos dados. Esse é o Gargalo de von Neumann, termo atribuído a John Backus em sua palestra do Prêmio Turing de 1977 [Couto 2017]. O problema se agravou porque a velocidade da CPU e o tamanho da memória evoluíram em ritmo superior ao da taxa de transferência entre eles [Couto 2017]. Uma forma de reduzi-lo é inserir uma cache entre a CPU e a memória principal e separar as caches de instruções e de dados, o que se chama arquitetura Harvard modificada [Couto 2017].

Figura 1. Máquina original de von Neumann.

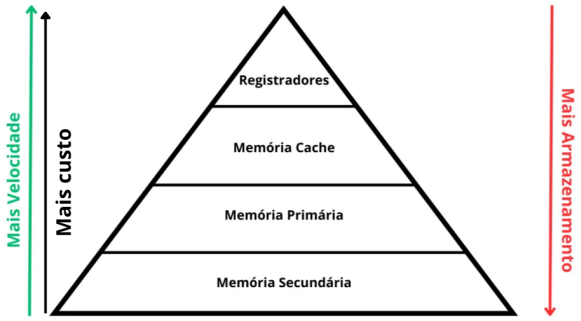


Fonte: [Tanenbaum e Austin 2013]

3.2. Hierarquia de Memória e Princípio da Localidade

Para conciliar custo, capacidade e velocidade, a memória é organizada em uma hierarquia, tradicionalmente representada como uma pirâmide que abrange desde os níveis mais rápidos e restritos até os de maior capacidade e menor velocidade, sendo composta por registradores, memória cache, memória principal e armazenamento secundário.

Figura 2. Hierarquia de memória.



Fonte: adaptado pelos autores(2026) de [Maués e Nascimento 2025]

Os níveis dessa hierarquia e suas principais características funcionais e tecnológicas são descritas da seqguinte forma por [Maués e Nascimento 2025]:

- **Registradores**: localizados no interior da CPU, constituem o nível mais rápido, de menor latência e de capacidade mais restrita da hierarquia, sendo responsáveis por armazenar operandos e instruções imediatamente utilizados pelo processador.
- **Memória Cache**: atua como um nível intermediário entre os registradores e a memória principal (RAM). Sua principal função é armazenar dados e instruções acessados com maior frequência para reduzir a latência de acesso.
- **Memória Principal (RAM)**: memória do tipo volátil e de capacidade intermediária que mantém as informações de forma temporária durante a execução dos programas e do sistema operacional. Apresenta menor velocidade em comparação aos níveis superiores, contudo oferece capacidade substancialmente maior a um custo inferior por bit.
- **Memória Secundária**: composta por dispositivos não voláteis como discos rígidos (HDD) e unidades de estado sólido (SSD), caracteriza-se pela alta capacidade de armazenamento e pelo baixo custo. É responsável pela retenção permanente de dados, arquivos e do sistema operacional.

O funcionamento eficiente dessa hierarquia apoia-se no Princípio da Localidade de Referência, que define a tendência de um processador acessar o mesmo conjunto de locais de memória. Esse princípio subdivide-se em:

- **Localidade Temporal**: refere-se à tendência de um dado ou instrução recentemente acessado ser requisitado novamente em um curto intervalo de tempo. Exemplos incluem funções, métodos, laços de repetição, contadores e variáveis modificadas com frequência [Ferreira 2015].

- **Localidade Espacial**: corresponde à tendência de itens armazenados em endereços de memória adjacentes ao item recém-acessado serem requisitados em seguida. Nessa categoria encontram-se partes procedurais e sequenciais do programa, além de estruturas de dados como vetores, matrizes e listas [Ferreira 2015].

3.3. Funcionamento e Estratégias na Memória Cache

Sem cache, cada ciclo de instrução exigiria ao menos um acesso à memória principal e, como a DRAM é mais lenta que a execução da instrução, o processador ficaria ocioso [Ferreira 2015]. A cache atenua esse descompasso armazenando cópias das instruções e dos dados mais usados [Stallings 2010, Ferreira 2015]. Quando a CPU requisita uma palavra, ocorre um acerto (*cache hit*) se ela está em alguma linha da cache, sendo entregue ao processador em um tempo T_{acerto}; caso contrário, ocorre uma falha (*cache miss*), e o dado é buscado nos níveis inferiores, com uma penalidade P_{falha} correspondente à latência da DRAM e ao trânsito no barramento. Para reduzir falhas futuras, a transferência ocorre em blocos de palavras adjacentes, aproveitando a localidade espacial [Stallings 2010, Ferreira 2015].

O efeito da cache pode ser medido pelo tempo médio de acesso à memória (AMAT) que, com base no modelo de dois níveis de [Stallings 2010], é dado por:

$$T_{med} = T_{acerto} + (1 - h) \times P_{falha} \tag{1}$$

, em que h é a taxa de acerto e $(1 - h)$ é a taxa de falhas. Com valores hipotéticos $T_{acerto} = 5$ ns (ordem de grandeza de uma cache L2), $P_{falha} = 60$ ns e $h = 0,95$, tem-se $T_{med} = 5 + 0,05 \times 60 = 8$ ns. A penalidade de falha é doze vezes o tempo de acerto, mas o tempo médio fica próximo ao da cache; ainda assim, as falhas, que ocorrem em apenas 5% dos acessos, respondem por 3 dos 8 ns ($37,5\%$).

Os projetos de cache combinam algumas estratégias. Na hierarquia multinível (L1, L2 e L3), a L1, interna ao processador, é a menor e mais rápida; a L2 é intermediária; e a L3, compartilhada entre os núcleos, é a última barreira antes da DRAM, de modo que a capacidade cresce para os níveis inferiores, enquanto a velocidade e o custo por bit crescem para os superiores [Stallings 2010, Ferreira 2015]. Nas caches separadas, a divisão da L1 em instruções (L1I) e dados (L1D) elimina a disputa entre a busca de instruções e a unidade de execução no pipeline, concretizando a arquitetura Harvard modificada da Seção 3.1, embora a cache unificada tenda a ter taxa de acerto maior para o mesmo tamanho [Stallings 2010].

Quanto ao mapeamento de endereços, o mapeamento direto associa cada bloco a uma única linha, sendo simples e barato, mas sujeito a faltas por conflito; o totalmente associativo permite qualquer linha, sem conflitos, mas exige comparar as etiquetas de todas elas; e o associativo por conjuntos de k vias combina simplicidade e flexibilidade [Stallings 2010]. Na política de escrita, o *write-through* grava na DRAM a cada escrita, mantendo os dois níveis consistentes, mas gera tráfego que pode saturar o barramento; o *write-back* grava apenas na cache, marca a linha com um bit de modificação (*dirty bit*) e atualiza a DRAM só quando o bloco é descartado [Stallings 2010]. Ferreira [Ferreira 2015] relata ganhos de até 15% em processadores Cyrix e de cerca de 30% em processadores IBM com o *write-back*. Por fim, quando o conjunto está cheio, algoritmos

como LRU (descarta o bloco menos recentemente usado, preservando a localidade temporal), FIFO ou aleatório escolhem o bloco a ser substituído [Stallings 2010].

3.4. Estratégias no Armazenamento Secundário

Em geral, o armazenamento secundário apresenta latência bem superior à das memórias voláteis, por depender de meios físicos para a gravação de dados [de Oliveira e Shin-Ting 2019, Stallings 2010]. Os HDDs são os principais armazenadores de longo prazo por terem um baixo custo e grande capacidade. Porém, para ler um dado, o braço de leitura precisa ser posicionado para pesquisa (*seek time*), que varia entre 4 e 10 ms, e o disco deve girar até que o setor seja acessado (latência rotacional), que varia entre 3 e 7 ms [Maziero et al. 2015]. Essa característica torna o acesso custoso e gera um tempo mais lento do que o acesso à DRAM. Assim, uma das grandes dificuldades de projeto é lidar com E/S.

Uma estratégia é o uso de memória virtual: a partir da necessidade de mais memória RAM do que a disponível por parte dos programas, o sistema operacional divide o espaço de endereçamento em páginas de tamanho fixo. A página é carregada a partir do disco, conforme necessário, para uma área chamada *swap* [de Souza e da Silva 2023]. Esse processo corre o risco de gerar o *thrashing*, que ocorre quando o conjunto de trabalho (*working set*) é maior que a RAM e, consequentemente, a maior parte dos ciclos é consumida na transferência de páginas entre o disco e a memória, o que prejudica o sistema [de Souza e da Silva 2023].

Figura 3. Tabela comparativa das estratégias de mitigação do Gargalo de Von Neumann

Estratégia	Tecnologia	Impacto na latência	Como amenizar o gargalo	Principal limitação
Cache: Mapeamento Associativo por Conjuntos	Divisão da cache em conjuntos de k-vias flexíveis.	Baixa (acesso rápido com controle de colisões).	Otimiza a taxa de acerto (*hit rate*) e reduz o AMAT com custo moderado de hardware.	Exige dimensionamento rigoroso para evitar contenção de linhas em aplicações concorrentes. [Stallings 2010, Ferreira 2015].
Cache: Política de Escrita	Write-Back (escrita direcionada apenas à cache, atualizando a DRAM tardiamente com *dirty bit*).	Muito Baixa para operações de gravação.	Evita o tráfego excessivo e saturação instantânea do barramento externo da memória principal.	Risco de perda de dados em caso de falha de energia antes da sincronização com a DRAM. [Stallings 2010, Ferreira 2015].
Armazenamento: Memória Virtual e Paginação	Divisão lógica em páginas fixas e uso de área de troca (*swap*).	Elevada (quando ocorre *page fault* e acesso ao disco).	Expande o espaço lógico de enderecamento além da capacidade física restrita da RAM.	Risco de degradação crítica de desempenho se o *working set* exceder a RAM (*thrashing*). [de Souza e da Silva 2023].
Armazenamento: Subsistemas Híbridos (BCache / DMCache)	Uso transparente de unidades SSD como cache de blocos para HDDs.	Moderada (elimina o *seek time* mecânico em acessos frequentes).	Amortece o abismo de latência entre a DRAM e os discos magnéticos em leituras/escritas aleatórias.	Custo adicional de hardware e necessidade de gerenciamento de desgaste (*wear leveling*) na *flash*. [Maziero *et al.* 2015].

Fonte: Elaborado pelos autores (2026), com base em [Stallings 2010, Ferreira 2015, de Souza e da Silva 2023, Maziero et al. 2015].

Os SSDs são mais rápidos que os HDDs pois utilizam memória NAND *flash*. Porém, não são vistos para substituir os HDDs para conjuntos massivos de dados, pois possuem custo mais alto por gigabyte, além de uma capacidade menor. Uma alternativa é utilizar o SSD como uma cache transparente dos dados mais acessados do HDD [Maziero et al. 2015]. No Linux, existem soluções como *BCache* e *DMCache*. O *BCache* trabalha na camada de blocos, independente do sistema de arquivos, e possui a vantagem de permitir acesso aleatório e ignorar solicitações estritamente sequenciais. O *DMCache* utiliza o *device-mapper* e exige partições dedicadas; além disso, permite três modos de escrita: *write-back* (apenas na cache), *write-through* (na cache e no dispositivo subjacente ao mesmo tempo) e *write-around* (diretamente no dispositivo subjacente) [Maziero et al. 2015]. A Figura 3 compara as estratégias de armazenamento secundário com as de cache.

4. Análise Crítica e Gargalos Arquiteturais

4.1. O Descompasso Físico e os Limites da Escalabilidade

Com o avanço das tecnologias de processadores, as memórias principais não acompanharam este avanço. Isto resulta em casos nos quais a CPU entra em modo de espera (wait time) para aguardar a chegada dos dados [Stallings 2010, Tanenbaum e Austin 2013]. O problema ocorre devido ao fato de que a DRAM armazena informações em capacitores, que ao receberem um bit, perdem carga ao longo do tempo, exigindo ciclos periódicos de atualização (*refresh*). Já a SRAM da cache utiliza flip-flop para armazenamento, não precisa realizar refresh, porém requer mais transistores para armazenar um bit, resultando em maior custo de implementação e menor densidade de integração [Stallings 2010]. Desta forma, a memória cache é rápida, porém com pequena capacidade e alto custo, sendo a memória principal de maior capacidade, porém mais lenta. (Figura 4).

O aumento da frequência de operação também fez surgir limites físicos: a partir de 2001, o aquecimento causado pela redução espacial dos transistores não podia ser dissipado utilizando-se refrigeração convencional. Isto fez com que o aumento do clock se tornasse difícil, o que levou os fabricantes a resolverem este impasse utilizando-se de processadores multicore [Ferreira 2015]. Com vários núcleos, o barramento tende a se tornar um ponto de contenção arquitetural. Para contornar essa situação, foi utilizada uma cache L3 compartilhada entre todos os núcleos (microarquitetura Nehalem). Esta cache atua inclusivamente sobre as caches L1 e L2 de cada núcleo, e evita que cada núcleo verifique a cache privada dos outros núcleos, o que aumenta consideravelmente o rendimento do sistema como um todo [Ferreira 2015].

4.2. Síntese dos Trade-Offs da Hierarquia de Memória

A Figura 4 apresenta uma síntese comparativa dos níveis que compõem a hierarquia de memória, destacando as ordens de grandeza típicas de latência, as capacidades ilustrativas e os principais compromissos de projeto associados a cada tecnologia.

Figura 4. Tabela comparativa dos níveis da hierarquia de memória e trade-offs arquiteturais

Nível da Hierarquia	Tecnologia e Localização	Ordens de Grandeza de Latência	Valores Ilustrativos de Capacidade	Trade-offs Arquiteturais e Impacto no Desempenho
Registradores	Circuitos de armazenamento integrados ao núcleo do processador.	Tipicamente menor que 1 ns	Centenas de bytes (32 a 64 registradores)	Menores e mais velozes unidades de memória [Miletto *et al.* 2014]; capacidade restrita pelo custo por bit e espaço físico [Ferreira 2015].
Cache L1	Memória SRAM integrada, dividida em instruções (L1I) e dados (L1D).	Aproxima-damente 0,5 a 1,5 ns	Dezenas de KB por núcleo (16 KB a 256 KB)	A divisão elimina a disputa entre busca e execução [Stallings 2010]; tamanho reduzido limita grandes blocos [Ferreira 2015].
Cache L2	Memória SRAM integrada, geralmente privada por núcleo.	Aproxima-damente 3 a 10 ns	Centenas de KB a alguns MB (256 KB a 1 MB)	Maior capacidade que a L1 para filtrar acessos externos; latência proporcionalmente superior [Stallings 2010, Ferreira 2015].
Cache L3	Memória SRAM integrada, compartilhada entre múltiplos núcleos.	Aproxima-damente 10 a 20 ns	Dezenas de MB (2 MB a 12 MB ou superior)	Arquitetura inclusiva (ex.: Nehalem) evita tráfego de *snoop* entre núcleos e otimiza o rendimento [Ferreira 2015].
Memória Principal	Memória DRAM conectada ao controlador de memória.	Aproxima-damente 50 a 100 ns	4 GB a 64 GB (ou superior)	Grande capacidade volátil; exige circuitos de *refresh* [Stallings 2010]; a limitação do canal de dados caracteriza o Gargalo de Von Neumann [Couto 2017].
Armazenamento Secundário	SSD (NAND flash) ou disco rígido magnético (HDD).	SSD/HDD: microsse-gundos a milissegundos	Centenas de GB a Terabytes	Não volátil e de menor custo por bit; faltas de página penalizam severamente o desempenho [Couto 2017, de Oliveira e Shin-Ting 2019, Stallings 2010].

Fonte: Elaborado pelos autores (2026) com base em [Couto 2017, Ferreira 2015, Hennessy e Patterson 2017, Miletto et al. 2014, de Oliveira e Shin-Ting 2019, Stallings 2010].A capacidade das memórias e as ordens de grandeza de latência por nível são baseadas em [Hennessy e Patterson 2017].

Em síntese, a Figura 4 evidencia o compromisso fundamental da hierarquia: os níveis superiores (registradores e caches SRAM) priorizam velocidade e baixa latência, enquanto os inferiores (DRAM e armazenamento secundário) oferecem alta capacidade e persistência a um custo por bit viável. Como, em geral, quanto maior a capacidade de um nível, menor sua velocidade de acesso, o desempenho computacional global depende diretamente da eficiência dos mecanismos de mitigação do gargalo de von Neumann ao longo de toda a estrutura.

5. Considerações Finais

Neste artigo, foram revistas as principais estratégias existentes na hierarquia de memória para minimizar o Gargalo de von Neumann. A discrepância entre a velocidade da CPU e a latência da DRAM, amplificada pela barreira térmica do *clock* e pela contenção no barramento em sistemas *multicore* [Ferreira 2015, Tanenbaum e Austin 2013], é contornada pelo uso de caches multinível, que aproveitam a localidade para diminuir o tempo médio de acesso por meio de políticas de escrita como o *write-back*, o*write-through* e *write-around* [Stallings 2010, Ferreira 2015], além do uso de SSDs e subsistemas de cache de bloco no armazenamento secundário para atenuar a diferença de latência em relação aos HDDs [Maziero et al. 2015]. Contudo, ressalta-se que esse gargalo não é eliminado, mas apenas reduzido.

As tendências futuras na computação, impulsionadas principalmente pela inteligência artificial, revelam os limites das estratégias atuais. O processo de carregar os bilhões de pesos de um modelo a partir da memória é constante, e a transferência de dados consome a maior parte da energia, enquanto o cálculo representa apenas cerca de 10% da carga, segundo pesquisadores da IBM [Hess 2025]. Para combater esse cenário, surgiram novas soluções: a computação em memória utiliza a memória de mudança de fase (*Phase Change Memory* – PCM) para armazenar os pesos do modelo na resistividade do material, minimizando a transferência de dados. No entanto, devido à baixa durabilidade da PCM, essa abordagem limita-se à execução de modelos previamente treinados em GPUs. Outras inovações incluem a computação próxima à memória, como no processador IBM NorthPole, que dota cada núcleo de sua própria memória SRAM, e a óptica co-embalada, que estende a largura de banda da fibra óptica até a borda do chip [Hess 2025]. Como a arquitetura de von Neumann continua fundamental para a computação de propósito geral e para operações de alta precisão, projeta-se o advento de sistemas híbridos que combinem processadores de von Neumann e não von Neumann [Hess 2025].

Referências

Couto, C. (2017). Arquitetura de von neumann. Material didático / Notas de aula.

de Oliveira, J. R. e Shin-Ting, W. (2019). Sistema de memória: Tecnologia. Notas de aula / Tópico 5, DCA–FEEC–Unicamp. Agosto de 2019.

de Souza, L. A. e da Silva, L. R. M. (2023). A evolução da gerência de memória em sistemas operacionais: da memória física à virtualização. *A sinfonia tecnológica: relação entre ciência e sustentabilidade*, 2(2).

Ferreira, L. V. R. (2015). Avanços na arquitetura de memo´ria cache. *UNESP - IBILCE*.

Hennessy, J. L. e Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6 edition.

Hess, P. (2025). Why a decades old architecture decision is impeding the power of ai computing. Disponível em: `https://research.ibm.com/blog/why-von-neumann-architecture-is-impeding-the-power-of-ai-computing`.

Maués, J. C. d. S. e Nascimento, E. L. B. d. (2025). Evolução da hierarquia de memória: impactos na arquitetura de sistemas computacionais e nas tendências de inovação tecnológica. *Cuadernos de Educación y Desarrollo*, 17(12):e10288.

Maziero, C., Dos Santos, L., e Bona, L. C. (2015). Avaliação de caches em dispositivos de armazenamento secundário com ssds. *Environmental Science & Technology Innovation*.

Miletto, E. M., Gonsales, A. D., e Okuyama, F. Y. (2014). O que é informática. *Desenvolvimento de software I: conceitos básicos. 1ª ed. Porto Alegre: Bookman*, pages 1–4.

Stallings, W. (2010). *Arquitetura e Organização de Computadores*. Pearson Prentice Hall, São Paulo, 8 edition.

Tanenbaum, A. S. e Austin, T. (2013). *Organização Estruturada de Computadores*. Pearson, 6 edition.